

Санкт-Петербургский политехнический университет Петра Великого
Институт машиностроения, материалов и транспорта
Высшая школа автоматизации и робототехники

КУРСОВАЯ РАБОТА

**Разработка игры в жанре «bullet hell» на языке C++ с
использованием библиотеки Raylib**
по дисциплине «Объектно-ориентированное программирование»

Выполнил студент
гр. 3331506/30401

Ковалев П. В.

Руководитель
Доцент

Ананьевский М. С.

«____» _____ 2026г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

Введение	3
Глава 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	5
1.1. Жанр bullet hell: особенности и типовые механики	5
1.2. Обзор существующих игровых движков и библиотек для 2D-игр	5
1.3. Выбор языка программирования и библиотеки	6
1.4. Требования к разрабатываемой игре	7
1.5. Постановка задачи	8
Заключение	9

ВВЕДЕНИЕ

Актуальность работы определяется необходимостью создания лёгких, переносимых и при этом достаточно выразительных игровых движков для обучения основам разработки компьютерных игр. Жанр «bullet hell» предъявляет повышенные требования к производительности и управлению большим количеством динамических объектов (снарядов), что делает его показательной задачей для отработки эффективных алгоритмов обработки данных, коллизий и событийного программирования. Библиотека Raylib, представляющая собой минималистичный, но полнофункциональный API для 2D- и 3D-графики, оконного управления и ввода, позволяет создавать игры без использования тяжёлых коммерческих движков (Unity, Unreal Engine) и при этом сохранять контроль над низкоуровневыми аспектами: циклами обновления, памятью, временем. Таким образом, разработка игры в жанре bullet hell на C++ с использованием Raylib является актуальной учебно-исследовательской задачей.

Новизна работы заключается в разработке событийно-ориентированной системы описания уровней, позволяющей гибко задавать временные последовательности атак (в том числе создавать вложенные события, когда одна атака порождает другие), а также в применении стандарта C++20 (в частности, `std::format` для форматирования строк интерфейса) и интеграции с кроссплатформенной системой сборки Premake/CMake. Такой подход упрощает добавление новых уровней и типов атак без изменения основного игрового цикла.

Цель работы: разработать на языке C++ с использованием библиотеки Raylib игру в жанре bullet hell, поддерживающую несколько уровней с различными паттернами атак и управление игроком с клавиатуры.

Объект исследования: методы организации игрового цикла, событийно-ориентированного планирования атак, обработки коллизий и отрисовки в реальном времени.

Предмет исследования: применение возможностей C++20 (включая стандартную библиотеку, контейнеры, алгоритмы, `std::format`) и библиотеки Raylib для реализации аркадной игры.

Задачи работы:

1. Провести обзор существующих решений и выбрать технологический стек.
2. Спроектировать архитектуру приложения: объектную модель игрока и снарядов, систему событий.

3. Реализовать базовые механики: движение игрока (клавиши WASD/стрелки), нормализацию скорости, проверку столкновений (круг–прямоугольник), учёт неуязвимости после попадания.
4. Разработать событийную систему для описания атак уровней, позволяющую создавать сложные последовательности (линии, лучи, взрывы, веера) и динамически добавлять новые события.
5. Создать два демонстрационных уровня (обучающий и сложный) с использованием этой системы.
6. Провести анализ полученных результатов.

Практическая значимость работы заключается в том, что разработанный код может служить основой для дальнейшего развития (добавления новых типов атак, звукового сопровождения, редактора уровней) или использоваться в учебных целях при изучении C++ и игрового программирования.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1. Жанр **bullet hell**: особенности и типовые механики

Жанр компьютерных игр «bullet hell» характеризуется большим количеством снарядов на экране, сложными и часто симметричными паттернами их движения, а также высокими требованиями к реакции игрока. В отличие от классических аркадных шутеров, где основное внимание уделяется поражению врагов, в bullet hell важнейшим навыком становится уклонение от большого числа одновременно движущихся снарядов.

Типовые механики, встречающиеся в играх этого жанра:

- **Плотные линейные залпы** — снаряды движутся параллельными рядами с небольшим интервалом.
- **Лучи и вереницы пуль** — пули выпускаются последовательно по одной траектории, создавая «нитку».
- **Круговые и веерные залпы** — снаряды разлетаются из центра под равными углами.
- **Прицельные снаряды** — пуля направляется в текущее положение игрока на момент выстрела.
- **Бомбы (медленные крупные снаряды)** — после достижения определённой точки взрываются, порождая новые снаряды.
- **Сложные паттерны** — комбинации перечисленных элементов, синхронизированные во времени.

Реализация таких механик требует от разработчика эффективного управления динамическими массивами (постоянное добавление и удаление снарядов), точных и быстрых алгоритмов проверки столкновений, а также удобного способа описания последовательностей атак.

1.2. Обзор существующих игровых движков и библиотек для 2D-игр

Для создания 2D-игр на C++ существует несколько популярных библиотек и фреймворков. Наиболее распространённые из них представлены в таблице 1.1.

Сравнение библиотек для 2D-игр на C++

Библио-тека	Лицензия	Аппаратное ускорение	Кроссплатформенность	Сложность освоения
SFML 2.x	zlib	Да (OpenGL)	Win / Linux / Mac	Низкая
SDL2	zlib	Да (OpenGL / Direct3D)	Win / Linux / Mac / Android / iOS	Средняя
Raylib	zlib	Да (OpenGL)	Win / Linux / Mac / Android / Web	Низкая
GLFW + OpenGL	zlib/libpng	Да (OpenGL)	Win / Linux / Mac	Высокая

SFML предоставляет удобные классы для графики, звука, сети и ввода, но требует установки дополнительных библиотек и может быть избыточным для простых проектов. **SDL2** — низкоуровневая библиотека, дающая полный контроль, однако её использование сопряжено с большим объёмом кода для инициализации и управления ресурсами. **Raylib** позиционируется как библиотека для обучения и прототипирования: она сочетает простоту использования, хорошую документацию и достаточную производительность. В отличие от GLFW (которая лишь создаёт окно и контекст OpenGL), Raylib предоставляет готовые функции для рисования примитивов, работы с текстурами и шрифтами, а также встроенные функции для работы со временем и случайными числами.

1.3. Выбор языка программирования и библиотеки

Для разработки был выбран язык **C++20** по следующим причинам:

- Высокая производительность и контроль над памятью, необходимые для обработки большого количества снарядов (до нескольких сотен одновременно).
- Поддержка современного стандарта C++20, включающего `std::format` для удобного форматирования строк интерфейса и `std::span`, `std::ranges` для работы с контейнерами.
- Кроссплатформенность и наличие качественных компиляторов (GCC, Clang, MSVC).

Библиотекой для графики, ввода и оконного управления выбрана **Raylib**

(версия 4.0 и выше). Обоснование выбора:

- Лицензия zlib/libpng, позволяющая свободное использование в любых проектах.
- Отсутствие внешних зависимостей (кроме системных библиотек OpenGL, GLFW, ALSA и т.п.).
- Простой и интуитивно понятный API: для отрисовки спрайта достаточно одной функции `DrawTexturePro()`, а для обработки клавиш — `IsKeyDown()`.
- Встроенная поддержка работы со временем (`GetFrameTime()`), случайных чисел, математических функций.
- Активное сообщество и хорошая документация.

1.4. Требования к разрабатываемой игре

На основе анализа жанра и возможностей инструментов были сформулированы следующие функциональные и нефункциональные требования.

Функциональные требования:

1. Игра должна содержать меню выбора уровня с возможностью переключения между доступными уровнями.
2. Игрок управляет персонажем с помощью клавиш WASD или стрелок; скорость движения нормализована для равномерного перемещения по диагонали.
3. Игрок имеет показатель здоровья (HP); при столкновении со снарядом HP уменьшается, а игрок становится неуязвимым на короткое время (0,5 с).
4. Снаряды представляют собой круги, движущиеся с постоянной скоростью по заданной траектории. При выходе за границы игрового мира снаряды удаляются.
5. Уровни описываются через временные события (атаки), которые порождают снаряды. События могут создавать новые события (например, взрыв бомбы).
6. После завершения всех событий и исчезновения всех снарядов уровень считается пройденным, игра переходит в состояние `LEVEL_COMPLETED`.
7. При падении здоровья до 0 игрок проигрывает, состояние игры — `GAME_OVER`.
8. Должен быть предусмотрен режим отладки (включается клавишей F3),

отображающий дополнительную информацию (FPS, точное положение игрока, хитбоксы, количество коллизий).

Нефункциональные требования:

- Частота кадров должна быть не менее 60 FPS при 500 активных снарядах на современном оборудовании.
- Исходный код должен быть структурирован, разделён на логические модули (`game.cpp`, `render.cpp`, `main.cpp`).
- Сборка должна осуществляться с помощью системы CMake или Premake, поддерживаться в среде Linux.
- Интерфейс игры должен быть адаптивным к изменению размера окна.

1.5. Постановка задачи

На основе сформулированных требований необходимо разработать программную систему, реализующую игру в жанре bullet hell. В рамках курсовой работы требуется:

1. Спроектировать объектную модель, включающую классы `Player`, `Projectile`, `AttackEvent` и перечисление состояний игры (`GameState`).
2. Реализовать игровой цикл, поддерживающий переходы между состояниями `MENU`, `PLAYING`, `LEVEL_COMPLETED`, `GAME_OVER`.
3. Разработать механику движения игрока с нормализацией скорости и ограничением границами мира.
4. Реализовать систему событий, позволяющую задавать атаки в виде функций с привязкой к абсолютному времени уровня.
5. Создать библиотеку вспомогательных функций для часто используемых паттернов атак: `add_beam_events`, `spawn_circular_burst`, `spawn_bullet_line`, `add_fan_attack`.
6. Разработать два демонстрационных уровня: «Beginning» (обучающий) и «Way» (с более сложными паттернами).
7. Реализовать отрисовку с преобразованием мировых координат в экранные (с сохранением пропорций и центрированием).
8. Подготовить инструкцию по сборке и запуску на платформе Linux.

Решение перечисленных задач позволит получить работающий прототип игры, демонстрирующий основные принципы организации игровой логики на C++ с использованием Raylib.

ЗАКЛЮЧЕНИЕ